

Data Model & Constraint Reasoning

The database is the source of truth. Every safety guarantee is enforced by a DB constraint **and** application logic — never app logic alone.

Tables

Table	Purpose
Doctor	A practitioner; owns availability templates and slots.
AvailabilityTemplate	Weekly recurring availability (day-of-week, start/end, slot length). Limited Sunday hours are just a narrower window.
AppointmentSlot	A concrete, bookable time slot (materialized from templates). The row that gets locked during booking.
Reservation	A patient's hold on a slot, with a TTL (<code>reservedUntil</code>).
Payment	A charge attempt against a reservation, with a stable idempotency key.
WebhookEvent	Ledger of received provider events — the storage-layer idempotency guard.
EventLog	Append-only log of every state transition (audit trail).

Status enums

- **SlotStatus:** `Available` → `Reserved` → `Paid`, plus `Expired`, `Cancelled`.
- **ReservationStatus:** `Active` → `Paid` / `Expired` / `Cancelled`.
- **PaymentStatus:** `Pending` → `Approved` / `Rejected` / `Refunded`.

Legal transitions are enforced in code by the state machine (`src/domain/stateMachine.ts`).

Constraints and why they exist

Constraint	Table	Why
<code>@@unique([doctorId, startsAt])</code>	AppointmentSlot	A doctor can't have two slots at the same instant; also makes slot materialization idempotent (re-runs can't duplicate).

Constraint	Table	Why
<code>no_overlap_per_doctor</code> (GiST exclusion, <code>btree_gist</code>)	AppointmentSlot	A doctor can never have two overlapping slots — enforced by the DB, not app logic.
<code>Reservation.slotId @unique</code>	Reservation	At most one reservation per slot — the core anti-double-booking guard.
<code>one_live_slot_hold</code> (partial unique index)	AppointmentSlot	Documents/guards that a slot holds at most one live (<code>Reserved</code> / <code>Paid</code>) claim.
<code>Payment.idempotencyKey @unique</code>	Payment	A retried "start payment" can't create a second charge.
<code>@@unique([provider, providerEventId])</code>	WebhookEvent	Duplicate/retried webhooks are a no-op — confirm once, charge once.

Concurrency model

Booking uses a raw `SELECT ... FOR UPDATE` inside a Prisma interactive transaction: concurrent callers block on the slot row, then read the updated status and lose cleanly. The expiry job and the payment webhook take the **same** row lock, which is what makes the payment-at-expiry race resolve to exactly one consistent outcome. See [WEBHOOKS.md](#) for the exception matrix.